Status	Finished
Started	Monday, 28 September 2026, 8:22 AM
Completed	Monday, 28 September 2026, 8:39 AM
Duration	16 mins 50 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
break	break is a keyword
IF	IF is not a keyword

Answer: (penalty regime: 0 %)

```
import sys

KEYWORDS = {
    "break", "case", "continue", "default", "defer", "else",
    "for", "func", "goto", "if", "map", "range",
    "return", "struct", "type", "var"
}

def check_keyword():
    word = sys.stdin.read().strip()
    if not word:
        return

    if word in KEYWORDS:
        print(f"{word} is a keyword")
    else:
        print(f"{word} is not a keyword")

if __name__ == "__main__":
    check_keyword()
```

	Input	Expected	Got	
✓	break	break is a keyword	break is a keyword	✓
✓	IF	IF is not a keyword	IF is not a keyword	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
`print(ValidParenthesis("()"))`	`true`
`print(ValidParenthesis("()[]{}"))`	`true`
`print(ValidParenthesis("]"))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s: str) -> bool:
    # Map each closing bracket to its corresponding opening bracket
    bracket_map = {")": "(", "}": "{", "]": "["}
    stack = []

    for char in s:
        # If the character is a closing bracket
        if char in bracket_map:
            # Pop the top element from the stack if it matches, else use a dummy character
            top_element = stack.pop() if stack else '#'

            # If the mapping doesn't match the stack's top element, it's invalid
            if bracket_map[char] != top_element:
                return "false"
        else:
            # It's an opening bracket, push it onto the stack
            stack.append(char)

    # If the stack is empty, return "true", otherwise "false"
    return "true" if not stack else "false"

```

	Test	Expected	Got	
✓	`print(ValidParenthesis("()"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("()[]{}"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("]"))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
import sys

def find_substring_index():
    lines = sys.stdin.read().split()
    if len(lines) < 2:
        return

    string1 = lines[0]
    string2 = lines[1]

    # find() returns the first occurrence index, or -1 if not found
    print(string1.find(string2))

if __name__ == "__main__":
    find_substring_index()
```

	Input	Expected	Got	
✓	thistest123string 123	8	8	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbccccccc

Answer: (penalty regime: 0 %)

```
import sys

def main():
    s = sys.stdin.read().strip()
    if not s:
        return

    # Parse character and multi-digit number pairs
    pairs = []
    i = 0
    n = len(s)

    while i < n:
        char = s[i]
        i += 1
        num_str = ""
        while i < n and s[i].isdigit():
            num_str += s[i]
            i += 1
        count = int(num_str) if num_str else 1
        pairs.append((char, count))

    # Calculate total length and resize in-place
    total_len = sum(count for _, count in pairs)
    chars = list(s)
    chars.extend([''] * (total_len - len(chars)))

    # Fill backwards from right to left
    write_idx = total_len - 1
    for char, count in reversed(pairs):
        for _ in range(count):
            chars[write_idx] = char
            write_idx -= 1

    print("".join(chars))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	a2b4c6	aabbbbccccccc	aabbbbccccccc	✓
✓	a12b3d4	aaaaaaaaaaaabbbdddd	aaaaaaaaaaaabbbdddd	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.

Example 1:

```
Input:
hello
Output:
olleh
```

Example 2:

```
Input:
Hannah
Output:
hannaH
```

Answer: (penalty regime: 0 %)

```
import sys

def reverse_string(s: list) -> None:
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1

def main():
    input_str = sys.stdin.read().strip()
    if not input_str:
        return

    # Convert string input into a list of characters (char[])
    char_array = list(input_str)

    # Reverse in-place
    reverse_string(char_array)

    # Print reversed result
    print("".join(char_array))

if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	hello	olleh	olleh	✓
✓	Hannah	hannaH	hannaH	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```
import sys


def is_palindrome(s: str) -> int:
    left, right = 0, len(s) - 1

    while left < right:
        # Move left pointer until an alphanumeric character is found
        while left < right and not s[left].isalnum():
            left += 1

        # Move right pointer until an alphanumeric character is found
        while left < right and not s[right].isalnum():
            right -= 1

        # Compare characters ignoring case
        if s[left].lower() != s[right].lower():
            return 0

        left += 1
        right -= 1

    return 1


def main():
    input_str = sys.stdin.read().strip()
    print(is_palindrome(input_str))


if __name__ == "__main__":
    main()
```

	Input	Expected	Got	
✓	A man, a plan, a canal: Panama	1	1	✓
✓	race a car	0	0	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
import sys


def process_keystrokes():
    input_data = sys.stdin.read().splitlines()
    if not input_data:
        return

    n = int(input_data[0].strip())

    for i in range(1, n + 1):
        if i >= len(input_data):
            break

        line = input_data[i]
        stack = []

        for char in line:
            if char == "^":
                if stack:
                    stack.pop()
            else:
                stack.append(char)

        if stack:
            print("".join(stack))
        else:
            print("-1")


if __name__ == "__main__":
    process_keystrokes()
```

	Input	Expected	Got	
✓	3 Hey ^ goooo^^glee^ lucke^y ^charr^ms ora^^nge^^^^	Hey google luckycharms -1	Hey google luckycharms -1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPangram(sentence):
    alphabet = "abcdefghijklmnopqrstuvwxyz"

    for char in alphabet:
        if char not in sentence:
            return "false"

    return "true"
```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result
arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar

Answer: (penalty regime: 0 %)

```
S = input()

username, domain = S.split('@')
domain_name, extension = domain.split('.', 1)

print(extension)
print(domain_name)
print(username)
```

	Input	Expected	Got	
✓	abcd@gmail.com	com gmail abcd	com gmail abcd	✓
✓	arvijayakumar@rajalakshmi.edu.in	edu.in rajalakshmi arvijayakumar	edu.in rajalakshmi arvijayakumar	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization
i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple
a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```
s = input().strip()
abbr = input().strip()

i = 0
j = 0

while j < len(abbr):
    if abbr[j].isdigit():
```

```
    if abbr[j].isdigit():
        # Abbreviation numbers cannot start with 0
        if abbr[j] == '0':
            print("false")
            break

        num = 0

        while j < len(abbr) and abbr[j].isdigit():
            num = num * 10 + int(abbr[j])
            j += 1

        i += num

        if i > len(s):
            print("false")
            break

    else:
        if i >= len(s) or s[i] != abbr[j]:
            print("false")
            break

        i += 1
        j += 1

else:
    if i == len(s):
        print("true")
    else:
        print("false")
```

	Input	Expected	Got	
✓	internationalization i12iz4n	true	true	✓
✓	apple a2e	false	false	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.